

02Protocol Whitepaper

Release date **September 2026**

Token **02Protocol** (symbol **02**) · BNB Smart Chain (BSC) · PancakeSwap V2

This edition is written against the contracts deployed on BSC mainnet. Every number here is either read from those contracts or derived from their constants and marked as approximate; nothing is projected, promised, or extrapolated as a return.

0. Our Story

02Protocol did not start from a blank page. It started from a mechanism that produced a real multiple, and then stopped working.

In 2021 the first OlympusDAO fork on BNB Smart Chain launched under the name **Taichi**. It opened around \$15 and reached about \$2,300 on its launch day — a multiple, not a rounding error, and the reason the design is worth studying at all. What happened next is what happened to every project built on that template: the mechanism was unsustainable. Its incentives only held while the price was rising, and the model had no answer for the moment new money stops arriving. That is a property of the template, not a verdict on a particular project or the people who ran it — and it is exactly why 02Protocol does not reuse it.

There was no second attempt that year. There were four years spent studying that template instead — the forks that followed, and then the second wave in the Chinese market — and writing down the exact point at which each one stopped working. The conclusion was not a better bonding curve. The conclusion was that a protocol should stop pretending to create yield it does not have, and should instead publish the arithmetic of its own flows — including the parts that are uncomfortable.

The one lesson kept from Taichi: a protocol can only be honest about what its code forces it to do. Everything downstream of that — the pricing, the vault, the fee paths — is an attempt to express that lesson in Solidity rather than in slogans. This document is written against the deployed contracts, and wherever a description and the deployed code disagree, the deployed code wins.

Who is behind this: the technical team is a group of **anonymous volunteers distributed around the world**. Nothing here asks to be taken on the reputation of a named founder, and the token has no team allocation and no developer reserve anywhere in it. That is deliberate — and it is also why this document keeps pointing at public state: every rule below can be read off the contracts and recomputed by anyone.

1. Why That Template Stopped Working

The pattern was remarkably consistent across projects, chains and languages. Four traits account for most of it, and each one is a direct target of a design decision in 02Protocol.

1.1 Public examples

Project	What it did	What happened next
Wonderland (\$TIME)	Reserve-currency mechanics with very high staking APY	The treasury and its leverage became the story; price fell far below any implied backing
KlimaDAO	Carbon-backed treasury, same rebase-and-bond structure	Yield was paid in the protocol's own token; the token's price fell faster than the treasury grew
Redacted Cartel (BTRFLY)	Governance-vote accumulation treasury	Structure became too complex to value; the original thesis was diluted away
TempleDAO	Intent-based treasury with a "safe" floor narrative	A floor described in one token while exposure lived in another did not hold in a drawdown

HectorDAO	Fork economics, aggressive emissions	Emissions outran any real inflow; the treasury could not buy back the supply it had printed
KLK, Webkey, NVB, Olympus Mine	The Chinese-market second wave of the same model	The same four traits, compressed into shorter cycles

1.2 The four traits behind it

1. **High APY masked inflation.** Rewards were paid by printing the protocol's own token, not by real revenue. The headline rate measured dilution, not income.
2. **Bonds were delayed sell pressure.** A bond buyer acquired tokens at a discount with a vesting schedule. Every vesting event was future supply looking for an exit.
3. **Treasuries mostly held their own token.** A balance sheet denominated in the asset it is meant to support cannot support it: when the price falls, the backing falls with it.
4. **(3,3) is not an equilibrium.** The cooperative outcome is individually irrational the moment the price starts falling. Selling first is always the better private choice, so it happens, and the equilibrium collapses.

1.3 What 02Protocol does differently

Trait	The 02Protocol response
Yield printed from nothing	All rewards are paid in BNB, sourced only from explicit fee and mint allocations: 7% of mint funds and 30% of sell-back fees to LP stakers, 3% of mint funds and 20% of sell-back fees to single-token stakers, plus their share of the 1% dividend portion of the 3% V2 trading tax (that 1% splits into roughly 0.67% to LP stakers and 0.33% to single-token stakers). There is no fixed APY and none is promised.
Bonds as delayed sell pressure	There are no bonds. Tokens are minted only through the curve, and only the user mint path can mint.

A treasury holding its own token	The vault holds BNB only. The protocol's redemption price is <i>vault BNB ÷ circulating 02 supply</i> — it never reads a market price to compute it.
(3,3) as an equilibrium claim	No coordination is assumed. The document states plainly that the backing price only moves one way — up (the sole exception is lending principal leaving the vault, see 4.4) — and that the V2 market price can move in either direction.

2. What 02Protocol Is

02Protocol is a token whose pricing is computed on-chain from a BNB vault and a circulating supply — nothing else. There is no oracle in the pricing path. The core relation is a division:

$$\text{backing price} = \text{vaultBNB} \div \text{circulatingSupply}$$

That number is the price at which the protocol itself will redeem 02 for BNB. It is a floor for the protocol's own redemption path; it does not fix the price at which 02 trades on the open market.

2.1 A single mint door

Minting has exactly one public entry point: the token contract's `mint()` — the function the Mint page of this dApp calls. There is no second contract and no second pricing, and that is deliberate: two doors with identical semantics would mean every future rule (limits, rationing, per-wallet treatment) has to be implemented twice, and the second one is easy to forget. Always mint from this dApp.

2.2 What 02Protocol is not

- No bonds. There is no discounted purchase with a vesting schedule.
- No rebases. Supply does not increase just because time passed.

- No team allocation. There is no reserved slice and no developer bucket.
- No vesting and no insider unlock. The only `_mint` call in the token contract sits inside the user mint path.
- No private or public sale. Tokens enter circulation by minting against the curve.

The supply ceiling is a constant, not a governance parameter: **MAX_SUPPLY = 21,000,000** tokens with 18 decimals. It cannot be raised by a vote, by the owner, or by an upgrade in the ordinary governance path.

3. How Mint Works

3.1 The price curve is continuous

The mint price follows a single straight line from **PRICE_MIN = 0.000000247946 BNB** to **PRICE_MAX = 0.000002479460 BNB** — exactly ten times the starting price. There are no steps, no tiers and no discrete bands:

$$\text{curvePrice} = \text{PRICE_MIN} + (\text{PRICE_MAX} - \text{PRICE_MIN}) \times \text{totalMintedBNB} \div \text{MINT_BNB_CAP}$$

The effective mint price is the greater of the curve price and the current backing price:

$$\text{mint price} = \max(\text{curvePrice}, \text{vaultBNB} \div \text{circulatingSupply})$$

Neither price constant has a setter. They are fixed at deployment and there is no function anywhere in the contract that can change them.

3.2 Mint rules

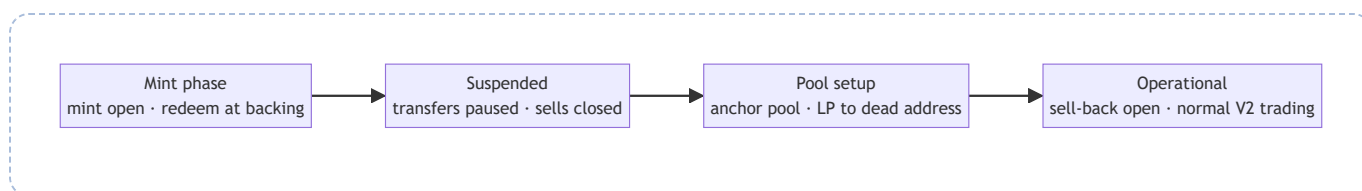
- **No minimum amount.** Any amount greater than zero is accepted.
- **Per-address cumulative cap: 0.1 BNB.** It is cumulative, not per transaction.

- **Anti-bot success draw.** An on-chain draw decides whether each mint lands, and the random success rate is adjusted as issuance progresses. On a miss the BNB is refunded in full and no quota is consumed.
- **Raise cap: 20 BNB.** Minting ends automatically when the cumulative minted BNB reaches `MINT_BNB_CAP`. A hard ceiling of **MAX_MINT_BNB = 500 BNB** sits above it as a constant bound; the factory raise cap in use is 20 BNB.
- **Minting switches on once.** It can be enabled a single time and cannot be switched off afterwards.

3.3 Where mint funds go

The split is hard-coded and not governable: **90% to the vault, 7% to LP-staking dividends and 3% to single-token staking dividends.** The 7% and 3% pre-fund the staking pools before staking opens. If a transfer to a pool fails, that part stays in the vault instead of being lost.

3.4 The three phases



Mint phase ends when the raise cap is reached. The protocol then enters a **suspended** state in which user transfers are paused, vault sell-back is closed, and the only permitted activity is the pool-setup window described in section 5. **Operational** phase opens vault sell-back and normal V2 trading and is entered by an explicit owner call.

4. The Vault & On-Chain Pricing

4.1 Two prices, two formulas

The mint price and the protocol's redemption price are *not* the same number, and this is not a rounding detail — they are different formulas:

$$\text{mint price} = \max(\text{curvePrice}, \text{vaultBNB} \div \text{circulatingSupply})$$

$$\text{redeem price} = \text{vaultBNB} \div \text{circulatingSupply}$$

If the supply or the vault is zero, the redeem price function returns PRICE_MIN. Sending 02 to the token contract address, or calling `sell()`, redeems at the backing price. The redeemed tokens are burned in full, always 100%.

4.2 Redemption fee

The fee is never zero. During the mint phase it decreases linearly from **30% to 5%** as the raise progresses (`FEE_START = 3000`, `FEE_END = 500`). After the mint phase it is a **fixed 5%**. The fee itself is split **50% to the vault, 20% to the single-token staking pool and 30% to the LP staking pool**. The vault's half is not withdrawn — it stays in and thickens the vault.

4.3 Honest math disclosure

Only 90% of mint funds reach the vault, while the curve price that a minter pays rises with every mint. These two facts together have a consequence that is worth stating precisely rather than implying it away: during the entire mint phase, the backing price stays *below* the current curve price. The vault is worth less than the sum the curve has been charging.

The ratio `backing ÷ curvePrice`, computed from the deployed constants (raise cap 20 BNB, a 10× curve), runs from approximately **90% at the start** down to approximately **35% at the end**. Put the other way round, the mint price is roughly **1.1× to 2.85× the protocol's own redemption price**.

Mint progress	backing ÷ curvePrice (approx.)	Meaning
0%	≈ 90%	Mint price ≈ 1.1× the redemption price
28%	≈ 51%	Mint price ≈ 2.0× the redemption price
50%	≈ 43%	Mint price ≈ 2.3× the redemption price

100%	≈ 35%	Mint price ≈ 2.85× the redemption price
------	-------	---

Table derived from the deployed constants: PRICE_MIN 0.000000247946 BNB, PRICE_MAX 0.000002479460 BNB, raise cap 20 BNB, 90% of mint funds to the vault.

At the end of the mint phase the vault holds **approximately 18 BNB** and the supply is **approximately 20.64 million** tokens, giving a backing price of approximately **0.000000872 BNB** against a curve price of **0.000002479460 BNB**.

So a participant who redeems at the backing price receives less than the curve price they paid, and the redeem fee (30% falling to 5%) applies on top of that. The channel that can pay more than the backing price is the **V2 market**, whose price is set by the pool and by whoever provides its liquidity — and a market price can move in either direction.

4.4 What actually makes the backing price rise

$$\text{backing price} = \text{vaultBNB} \div \text{circulatingSupply}$$

The numerator grows when new mints arrive (they add 90% of the BNB while the average price paid is climbing), when the V2 tax's 1% vault share flows in, when 50% of sell-back fees return to the vault, when the buyback runs, and when liquidation principal returns. The denominator can shrink because a redemption burns 100% of the redeemed tokens.

The backing price only moves one way: up. As long as anyone is using the protocol it is pushed higher — minting (the curve price sits above the backing price, so every mint lifts it), every V2 swap (1% to the vault plus a 0.5% burn), every redemption (100% of the redeemed tokens are burned while half the fee stays in the vault), liquidation principal returning, the buyback, and any ecosystem revenue added later.

The one thing that pushes it down is lending principal leaving the vault: the vault's BNB falls while the O2 supply does not move. When the loan is repaid, principal plus interest return to the vault and the backing price lands higher than before — which is exactly why borrowing is capped at 20% of (vault + already borrowed), and why it has to be repaid.

But the accounting has to be done properly: the BNB that leaves is not gone — it becomes a **receivable anchored to the borrower's collateral**. A borrower must post O2 worth at least **125%** of the loan (valued at the backing price), so every BNB lent out sits behind more value in O2 than it is worth, on top of the 20% cap on lending itself. If the borrower does not repay

within 7 days, anyone (typically a keeper bot watching the chain) can liquidate: the collateral is sold **at the V2 market price**, and only the portion that covers principal + interest is sold, with a minimum-proceeds floor (at least the **principal itself** when the collateral covers the whole debt; 85% of the backing price when it does not). If the market cannot meet that floor the whole liquidation **reverts** — the debt is not cleared, the collateral is not released, and the borrower can still repay normally.

The outcome is that **the vault only takes back its principal — it stays whole**, and it **never touches the borrower's surplus**: any unsold O2 is credited to the borrower's claimable balance and can be withdrawn at any time. There is **no penalty of any kind**; the only thing deducted is 1% of the market premium, paid to the keeper as gas compensation.

So the complete statement is: **the backing price never goes down**. While a loan is outstanding the on-chain `getCurrentPrice()` reading dips briefly, because that BNB has temporarily left the vault — but it has only changed from "BNB in the vault" into "principal receivable". Nothing is lost on net, and when it comes back, principal plus interest return to the vault and the backing price stands higher than before.

It only sits still when there is no volume, no redemption, no ecosystem revenue and no outstanding loan — which in practice is close to impossible. And it cannot mathematically reach zero: the backing price is $\text{vaultBNB} \div \text{circulatingSupply}$, the vault only shrinks through redemption (which burns the matching supply and pays out less than one backing price per token) and through lending (capped at 20%, and repayable), so neither number reaches zero and the quotient is always positive. The protocol does not, and cannot, generate BNB on its own.

5. Pool Setup on V2

There is no liquidity-migration event. No migration function exists in the deployed contracts, and the protocol never moves vault BNB into a trading pool to fund one.

What the contracts actually do is a short, ordered procedure:

- `LiquidityManager.addLiquidityAndStake()`, which adds 02 and BNB and automatically stakes the LP into LPReward.

ing depth. **Depth has to come from the market.** The protocol does not pretend the
hor pool is a liquidity guarantee, and it does not spend the vault to make the pool look

6. Two Ways to Exit

There are exactly two ways to turn 02 back into value, plus one behaviour that applies before the market exists.

6.1 The V2 market

Once the operational phase is open, O2 trades normally on PancakeSwap V2. A swap is subject to a **3% tax** ($TAX_RATE = 300 \text{ bps}$). The split is: **1% to the vault** ($vaultRateBps = 100$), **0.5% burned** ($burnRateBps = 50$), **0.5% to the Foundation** (paid in O2 and never sold on the market; this slice comes out of the dividend part), and the remaining **1% to dividends**, which is divided by $v2LpShareBps = 6667$ into approximately **0.67% to LP stakers and 0.33% to**

single-token stakers. The vault/burn/dividend split is governable; the 3% headline rate is not. Tax is collected in 02 and swapped to BNB once the accumulated tax reaches `SWAP_THRESHOLD = 1,000` tokens; if a swap fails the tokens stay and are retried later.

The tax is not limited to the pair this protocol created. **Every 02 pool is taxed:** a pair is recognised by its own `getReserves()` interface, so a parallel 02/BNB pool opened by anyone is taxed on both legs exactly like the official one — funding it costs 3%, and taking 02 back out of it costs 3% again. Nobody can obtain a tax-free venue by deploying a second pool, and the official pool is not treated specially. The only way a pool becomes tax-free is a deliberate, owner-only `setIsExcluded(pair, true)`, which exists for a partner pair whose own token contract charges the tax on its side instead. A venue that does not answer `getReserves()` (a custom or renamed AMM) is not recognised as a pool; in practice a fee-on-transfer token such as 02 cannot be listed on one without breaking that venue's own accounting. The scope itself has one governor switch, `setPoolTaxScope: 1` narrows the tax back to the pool this protocol created, `0` widens it to every pool again, and `2 / 3` freeze whichever scope is in force for good — irreversibly, like the vault-withdrawal seal. The protocol's own pool pays its 3% in every one of those modes, so no setting of this switch can switch the core tax off.

6.2 Vault redemption

Sending 02 to the token contract address, or calling `sell()`, redeems at the backing price. This is allowed during the mint phase and again once the operational phase has opened. Tokens are burned in full, and the redeem fee described in section 4.2 applies. The same honest arithmetic applies here: the redemption price is the backing price, which during the mint phase sits below the curve price a minter paid.

6.3 Transfers during the mint phase

In the mint phase a user-to-user transfer is not rejected — the entire transferred amount is **burned 100%**. This is not a tax that goes somewhere; the tokens cease to exist. During the suspended phase, user transfers are paused. In the operational phase a plain wallet-to-wallet transfer carries no tax at all.

7. 02 Stake

Staking has two separate contracts and two separate pools: a single-token pool (RewardDistributor) and an LP pool (LPReward). Both pay rewards in BNB. There is no fixed APY and none is promised. Both default to disabled and are switched on by the owner.

7.1 Lock terms and dividend weight

Lock term	Dividend weight	Effect
30 days	1	Baseline share of dividends
60 days	2	Twice the dividend share of a 30-day stake of the same size
90 days	3	Three times the dividend share of a 30-day stake of the same size

This is a *weight* difference, not a different headline rate. Two stakes of the same size receive dividend shares in the ratio 1 : 2 : 3 according to their lock term.

7.2 Caps and accrual

- Total single-token stake is capped at **30% of circulating supply** (MAX_STAKE_RATIO = 3000).
- **There is no cap on LP staking.**
- Rewards accrue **hourly** (REWARD_INTERVAL = 1 hour) as $\text{amount} \times \text{weight} \times \text{accumulatedRewardPerShare}$.
- A matured stake keeps accruing until it is redeemed: it keeps its share of the dividend denominator, so what it has already earned is neither lost nor paid out early while it sits unclaimed. Re-staking is a manual action; there is no auto-compounding.
- Early unstake is allowed only after **24 hours** locked, returns **principal only**, forfeits all pending rewards, and the forfeited rewards are redistributed among the stakers at that time.

7.3 Where the rewards come from

Reward source	Single-token pool	LP pool
Mint funds	3%	7%
Sell-back fees	20%	30%
V2 trading tax	its share of the 1% dividend part	its share of the 1% dividend part
Lending interest	only if governance sets <code>lendDividendBps > 0</code> (at launch <code>vaultShareBps = 10000</code> , so the whole interest goes to the vault)	its share once set

8. 02 Bank

02 Bank is an over-collateralised lending facility inside the protocol. Collateral is 02, valued at the protocol backing price — not at a market price.

Parameter	Value
Collateral	02, valued at the protocol backing price
Maximum loan-to-value	$\leq 80\%$ (enforced as $\text{valuation} \times 100 \geq \text{borrowBNB} \times 125$)
Borrowing cap	20% of $(\text{vaultBNB} + \text{totalBorrowed})$
Interest	1.5% per day, simple, accrued per second (<code>INTEREST_RATE_DAILY = 150 bps</code>)
Default term	7 days
Minimum borrow time	1 day

After the deadline, liquidation can be triggered by anyone. The liquidation sells **just enough** 02 on PancakeSwap V2 to cover principal plus interest, rounded up. The slippage floor is **the principal itself** whenever the collateral covers the whole debt, so a liquidation that succeeds can never leave the vault short; the flat 85%-of-the-backing-price floor applies only when the collateral cannot cover the debt. If the sale cannot clear its floor at all — a pool with no depth — the whole call **reverts**: the debt is not cleared and the collateral is not released, so a borrower can never walk away with a free loan. Any unsold collateral is written to `pendingCollateral` for the borrower to claim later.

Proceeds are allocated as follows: principal returns to the vault, while the interest and any surplus are split three ways — the backing vault, the staking dividend pools, and the buyback — and a **1% keeper fee** is paid. At launch `vaultShareBps = 10000`, so **100% of the interest deepens the vault**: the vault raises the backing price from the first block and needs no V2 depth, whereas the buyback has to buy 02 at a market price above the backing price and cannot execute while the anchor pool holds only a small amount of 02. The owner moves the split later, and the dividend part it releases then splits by `lpShareBps = 6667`. Lending defaults to disabled.

9. Tokenomics

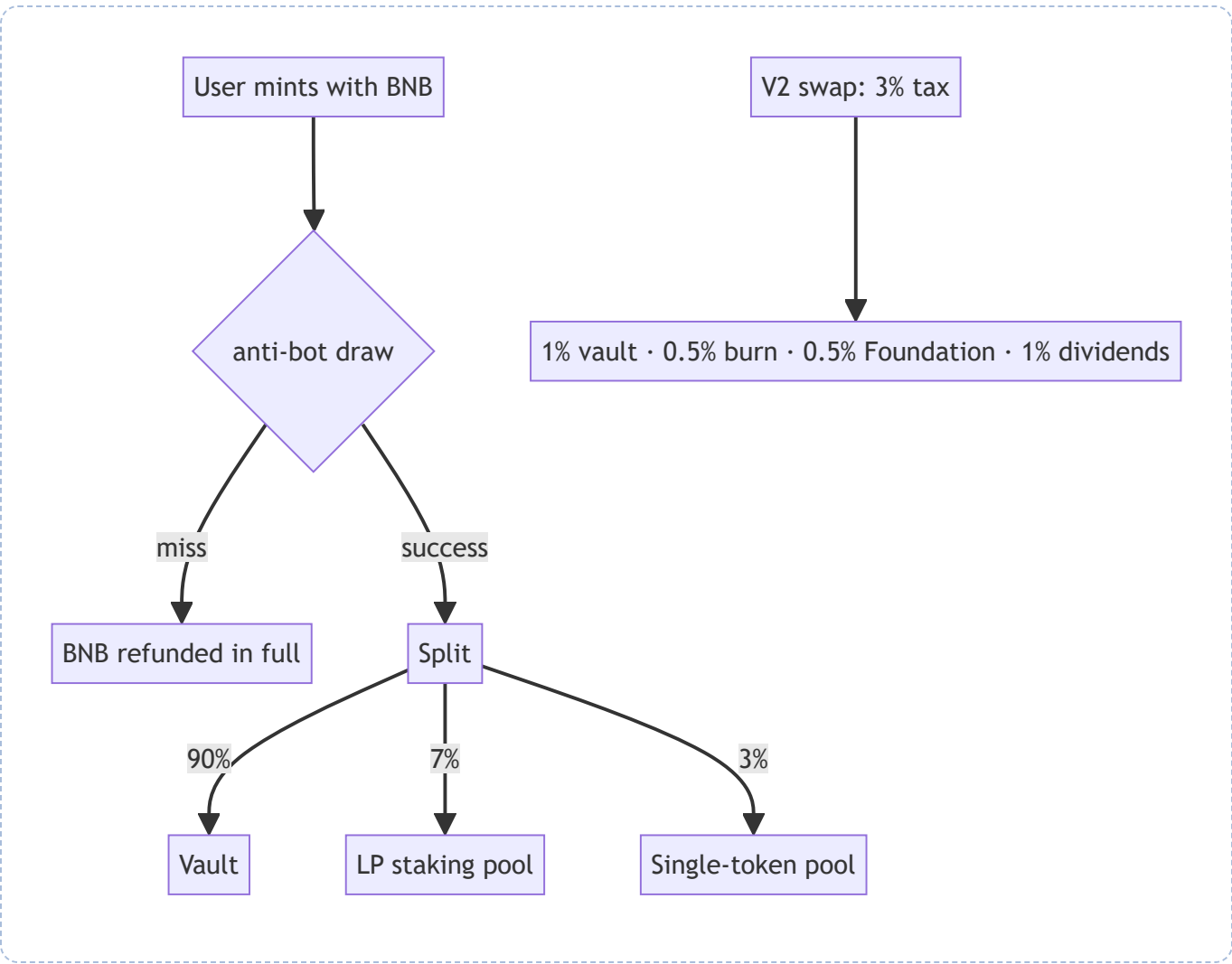
9.1 Supply

Property	Value
Name / symbol	02Protocol / 02
Decimals	18
Hard supply ceiling	21,000,000 (constant, not governable)
Team allocation	none
Reserve / vesting / sale	none — the only mint path is the user mint
Hard raise ceiling	<code>MAX_MINT_BNB = 500 BNB</code> (constant)

Factory raise cap in use	MINT_BNB_CAP = 20 BNB
Per-address cumulative cap	0.1 BNB

After minting has started, the Foundation may only *lower* the raise cap and the per-address cap. The worst case therefore shrinks, and the supply ceiling cannot be breached.

9.2 Fund flow



```

MINT (BNB) ----- anti-bot success draw
|                                     | miss -> full refund
|
+-- 90% -----> VAULT (backing price numerator)
+-- 7% -----> LP staking rewards pool
+-- 3% -----> Single-token staking rewards pool
  
```

SELL-BACK FEE (30% -> 5%, then fixed 5% of the redeemed value)

+-- 50% -----> VAULT

+-- 20% -----> Single-token staking rewards pool

+-- 30% -----> LP staking rewards pool

V2 SWAP TAX (3% of the swap, collected in 02, swapped to BNB at 1,000 tokens)

+-- 1% -----> VAULT

+-- 0.5% -----> BURN

+-- 0.5% -----> FOUNDATION (paid in 02, taken out of the dividend part)

+-- 1.0% -----> DIVIDENDS

+-- ~0.67% (2/3) -----> LP stakers

+-- ~0.33% (1/3) -----> Single-token stakers

9.3 Buyback

The token contract keeps a separate buyback pot (`buybackPool`), filled through `addBuyback()` — which receives whatever share of lending interest the split routes to it, and at launch that share is zero. With `buybackModeBurn = true`, the default, `buybackAndBurn()` buys 02 on PancakeSwap V2 and burns it, which is an explicit call rather than an automatic step, because the fill it gets depends entirely on the depth of the pool at that moment. If the mode is set to false the BNB is sent to `buybackSink` instead. The buyback pot is accounted separately from `vaultBNB`.

9.4 TwoSwap

TwoSwap is an optional built-in swap helper. Its fee ceiling is 50 bps and its default is `feeBps = 25`, i.e. **0.25%**. It is disabled by default. This is a separate fee, charged in addition to the 3% transfer tax — not instead of it.

10. What Makes the Backing Price Rise

The backing price is $\text{vaultBNB} \div \text{circulatingSupply}$. Two things can move it: the numerator can grow, and the denominator can shrink.

Driver	Effect on vault BNB	Depends on
--------	---------------------	------------

New mints (90% of funds)	Adds BNB to the vault while the average price paid is climbing	New capital entering the mint
V2 tax, vault share (1%)	Adds BNB continuously as swaps happen	Trading volume
Sell-back fees (50% of the fee to the vault)	Adds BNB, and keeps the other half in place	Redemption activity
Buyback	Buys and burns O2 by default, reducing supply	Buyback pot funding
Liquidation principal	Returns borrowed BNB to the vault	Lending activity
Redemption (burn)	Reduces circulating supply	Holders choosing to exit

Every row above depends on activity: with nobody acting, the backing price does not move. But as long as anyone is acting it only goes up — **the sole downward pressure is lending principal leaving the vault** (see 4.4). The tax-driven portion additionally depends on volume.

11. Exit Economics

There is no table here showing which mint timing is profitable, because the protocol cannot know that and will not claim it. What it can do is state what each exit channel pays.

Exit channel	What it pays	Costs
V2 market	Whatever the pool price is at the moment of sale — it can be above or below the backing price	3% V2 swap tax
Vault redemption	The backing price: $\text{vaultBNB} \div \text{circulatingSupply}$	Redeem fee 30% → 5% during mint, then fixed 5%

And the uncomfortable part, restated because it belongs in this section and not only in a footnote: during the mint phase the backing price is always below the curve price, with the ratio falling from approximately 90% to approximately 35%. A minter who exits through vault redemption during the mint phase therefore redeems below the price they paid, before the fee. The V2 market is the only channel that can pay more than the backing price, and only if someone is willing to buy there.

This is a description of the deployed arithmetic. It is not a prediction, and it is not advice.

12. Why the Protocol's Floor Cannot Reach Zero

The claim here is narrow and mathematical, and it is about the protocol's own redemption price, not about the market price of 02.

The redeem price is $\text{vaultBNB} \times 1e18 \div \text{circulatingSupply}$. As long as the vault holds any BNB and the circulating supply is non-zero, this quantity is strictly greater than zero. The vault is funded by 90% of mint funds, 50% of sell-back fees, 1% of every V2 swap and liquidation principal; redemptions themselves burn 100% of the tokens they redeem and the vault keeps half of the fee, so a redemption shrinks the numerator and the denominator together — and the net effect on this price is upwards, never downwards. The pricing function has no path that returns zero while those conditions hold.

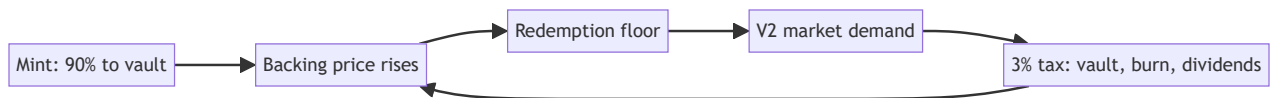
The permanent seal, once thrown, reinforces this at the contract level — and it has not been thrown yet (see section 14). Until it is thrown, the owner can withdraw vault BNB at any moment through a single `ownerWithdrawVault` call, with no notice period. `sealVaultWithdraw()` is a **one-way, irreversible** switch: once it is called, `ownerWithdrawVault` reverts forever, for everyone including the permission wallet, and `setLendingContract` is permanently locked as well, since the lending contract was the one remaining path that could pull from the vault. The seal requires the lending contract to be configured first. `emergencyWithdrawBNB` can only ever take BNB held *above* `vaultBNB`, never the vault itself.

Outside the vault, the "rescue" functions are deliberately fenced in. `ownerRescueToken` on the token contract refuses to move 02 itself; the two staking contracts refuse to move the token they hold on behalf of stakers (02 in single-token staking, the LP token in LP staking); the lending contract's `emergencyWithdrawTokens` refuses the collateral token; and `ownerRescueBNB` on each staking contract pays out only the BNB held *above* the pools it owes stakers

(rewardPool + claimableReserve, respectively lpRewardPool + lpClaimableReserve). The deployment-only upgrade hooks `initV9` / `initV10` / `initV11` are owner-only, so no third party can overwrite mint parameters the DAO has already set.

This guarantee is about the protocol's redemption price only. It does not mean the market price cannot fall, and it does not mean a holder can always obtain the backing price. It means the protocol's own pricing function has no zero outcome while the vault and supply are non-zero.

13. The Flywheel



Read the loop as a description of flows, not as a guarantee that it spins. Minting adds 90% of the proceeds to the vault, which raises the backing price. The backing price is the price at which the protocol redeems, which is one input to whether holders choose to hold or sell. Trading generates the 3% tax, whose 1% vault share and 0.5% burn feed back into the numerator and the denominator. Dividends paid in BNB to stakers are funded by the same flows.

Every arrow depends on someone choosing to act, and on someone providing liquidity for the V2 leg. With nobody acting, the flywheel does not turn; but once it turns, the backing price only moves up.

14. Roadmap

Item	Status
------	--------

Contracts deployed on BSC mainnet	Complete
On-chain self-check comparing implementation addresses against the deployment record	Complete
Mint phase	Not started
Third-party security audit	Not performed — pending
Vault sealing (<code>sealVaultWithdraw()</code>)	Pending
Migrating ownership to a Gnosis Safe with a community / threshold layout	Pending
Staking activation (single-token and LP pools)	Pending
Lending activation (02 Bank)	Pending
Governance activation (<code>governanceEnabled</code>)	Pending

There is no testnet in the public product. No dates are promised for any pending item. The audit line stays in this table as pending until an audit is actually completed — this document does not treat an audit as done.

15. Governance

Governance is split between `TwoDAO` (proposals) and `TwoFoundation` (the permission set). `governanceEnabled` defaults to false and is switched on once.

Parameter	Value
Right to propose	Voting power > 0, which requires a 90-day-locked single-token or LP stake

Voting weight, single-token	1 staked OZ = 1 vote
Voting weight, LP	An LP stake counts as 2× its OZ content
Notice / voting window	User proposals: chosen by the proposer, at least <code>MIN_USER_WINDOW = 1 day</code> and at most <code>MAX_NOTICE_PERIOD = 30 days</code> ; a Foundation proposal may use 0 (effective immediately). The interface default is 3 days
Pass threshold	<code>passBps = 5100</code> → 51%
Quorum	<code>quorumBps = 0</code> → quorum is disabled: there is no minimum participation, so a single address with voting power can satisfy "more than half of the votes cast" (the proposal still has to be executed by the Foundation multisig). This is a deliberate decision, not a pending item: the real gate is that a passed proposal still has to be executed by the Foundation multisig, which can also reject it
Execution	After <code>voteEnd</code> , Foundation proposals may be executed by anyone; a user proposal must be executed by the Foundation multisig, which can veto it.
Multisig	<code>TwoFoundation</code> implements M-of-N with <code>MAX_REQUIRED = 9</code> . Signers and the threshold can be changed on-chain without redeploying.
Referral rewards	Built into the contracts (a referrer earns OZ on the referee's staked amount, <code>referralBps ≤ 5000</code>), but <code>governanceEnabled = false</code> : nothing accrues and nothing is paid out today . It will be announced before it is switched on, and its design is fixed first.

15.1 Locked and changeable

Locked forever (no setter exists)

Changeable

Price curve constants (PRICE_MIN, PRICE_MAX) MAX_SUPPLY = 21,000,000 MAX_MINT_BNB = 500 Mint-phase split 90 / 7 / 3 TAX_RATE = 3%	V2 vault / burn split Dividend splits TwoSwap fee switch and rate (within the 0.5% cap) lendDividendBps Raise cap and per-address cap — Foundation only, and only downwards once minting has started
--	--

16. Relationship to 01 Protocol

02Protocol and 01 Protocol share a name pattern and a conviction about on-chain pricing, but the pricing machinery is different, and one row of the table below needs stating plainly.

Dimension	01 Protocol	02Protocol
Chain	Ethereum	BNB Chain (BSC)
Launch model	Fair launch through a bonding curve	Fair minting through a public bonding curve; no team allocation, no vesting, no reserve
Backing asset	Reserves held in yield-bearing sUSDS	The vault holds BNB. It is not used as trading capital and is not a source of trading depth
Supply	Fixed supply issued along the bonding curve	Hard ceiling of 21,000,000; no inflation-based incentives
Pricing model	Exponential bonding curve	Mint price follows a rising linear curve; the protocol redemption price is the backing price
Price consistency	Curve price and market price can differ	The two protocol prices are not the same number : the mint price is $\max(\text{curvePrice}, \text{backing})$ while the redemption price is only the backing price. During the mint phase the mint price sits above the redemption price.

Floor behaviour	Elastic — it can rise with yield and fall with redemptions	The backing price is $\text{vaultBNB} \div \text{circulatingSupply}$. A redemption burns supply and returns part of its fee to the vault, so it does not pull this price down. This says nothing about the V2 market price.
Reward source	Yield earned by the reserve asset	BNB paid from real fees and from part of the mint funds; no token printing
Governance	Progressive decentralisation	Multisig plus progressive community governance; governance is not enabled yet

01 and 02 do not price mint and exit at the same number. 02's mint price and its redemption price are different numbers for the whole mint phase, and sections 4 and 11 explain the gap rather than hide it.

17. Risks & Disclosures

This section exists because the mechanisms above have costs, and a document that hides them is not useful.

17.1 The mint premium

During the mint phase the backing price is always below the curve price — the ratio runs from approximately 90% down to approximately 35%, meaning the mint price is roughly $1.1\times$ to $2.85\times$ the protocol's own redemption price (section 4.3). A holder who exits through vault redemption during the mint phase realises less than they paid, before the redeem fee.

17.2 The redeem fee

Redemption is never free. It starts at 30% and falls linearly to 5% across the mint, then stays at a fixed 5% afterwards. Fees are deducted from the amount redeemed.

17.3 Anchor pool versus real liquidity

The pool the protocol seeds is a minimal price anchor, not a source of depth. Real trading depth must be provided by the market through the LiquidityManager. A thin pool means high slippage, and the protocol does not stand behind the pool with vault BNB. The interface additionally blocks trades when the estimated pool depth is below \$100, but that is a front-end guard only — there is no depth threshold in the contracts, and a direct contract call can bypass it.

17.4 Smart-contract risk

Code can contain defects that neither this document nor the deployment process has found.

17.5 Governance setup

Governance (`governanceEnabled`) is not yet switched on. Quorum is disabled, and the Foundation can veto a user proposal by declining to execute it.

17.6 No audit yet

A third-party security audit has **not** been performed. This document does not claim otherwise. The audit is listed as pending in the roadmap.

17.7 Market and liquidity risk

The V2 market price of O2 can move in either direction and is set by the pool and its liquidity providers. The backing price is a floor for the protocol's own redemption path only; it is not a floor under the market price, and it does not obligate anyone to buy at that level.

17.8 This is an experiment, not investment advice

O2Protocol is an experiment in on-chain, reserve-backed pricing. Nothing in this document is investment advice, a recommendation, or a promise of any outcome. Read the contracts, verify the addresses, and size any exposure accordingly.

18. Final Words

There is a roadmap in section 14, and it lists work that is still pending, including an audit that has not happened.

What survives is the one idea the project started with — that code sets the rules, and that the honest thing to do is publish what the code actually does even when the arithmetic is unflattering. The 90/7/3 split, the 30% → 5% redeem fee, the ratio falling from 90% to 35%, the unaudited implementation: all of it is written down here because a participant who understands these facts is making a real decision, and a participant who does not is making a guess.

02Protocol does not promise that the price rises. It promises that the rules are visible, that the pricing comes from a division anyone can recompute from public state, and that where a description and the deployed code disagree, this document follows the code.

02Protocol Whitepaper · September 2026 · This document describes deployed smart contracts on BNB Smart Chain. It contains no audit claim and no promise of profit. No third-party audit has been performed. Nothing here is investment advice. Verify contract addresses independently before interacting.